

EE332 Digital System Design

Dr. Yi Zeng
Assistant Professor,
Department of Electrical and Electronic Engineering
Office: S236, South Tower of COE
Email: zengyi@sustech.edu.cn

Chapter 8: Parameterized Design

- Goal: Design reuse
 - Ideally, we want to design some common modules that can be shared by many applications.
 - Since every application is different, it is desirable that a module can be customized to some degree to meet the specific need of an application.
 - Customization is normally specified by explicit or implicit parameters

260

Types of Parameters

- Width Parameters
 - The widths of data signals normally can be modified to meet different requirement.
 - The width parameters of a parameterized design specify the sizes (i.e., number of bits) of the relevant data signal.
- Feature Parameters
 - Specify the structure or organization of a design.
 - Defined on an ad hoc basis.
 - To include or exclude certain functionalities (i.e., features) from implementation or to select one particular version of the implementation

261

8.1 Generics

- The generic construct of VHDL is a mechanism to pass information into an entity and a component.
 - They are first declared in entity and component declaration and later assigned a value during component instantiation

```
entity para_binary_counter is
  generic (WIDTH: natural); 传入参数
  port (
    clk, reset: in std_logic;
    q: out std_logic_vector(WIDTH-1 downto 0)
  );
end entity para_binary_counter;
```

262

- After the declaration, the generic can be used in the associated architecture bodies.
- A generic cannot be modified inside the architecture body and thus functions like a constant
 - It is sometimes referred to as a generic constant.

263

```
architecture arch of para_binary_counter is
  signal reg, reg_next : std_logic_vector (WIDTH-1
    downto 0);
begin
  process (clk, reset) is
  begin
    if reset = '1' then
      reg <= (others => '0'); 由于width为可变参数, 使用others进行赋值
    elsif clk'event and clk='1' then
      reg <= reg_next;
    end if;
  end process;
  -- next-state logic
  reg_next <= reg + 1;
  q <= std_logic_vector(reg);
end architecture arch;
```

264

- To use the parameterized free-running binary counter in a hierarchical design, a similar component declaration should be included in the architecture declaration.
- The generic can then be assigned a value in the generic mapping section when a component instance is instantiated.
- Example of the use of generics

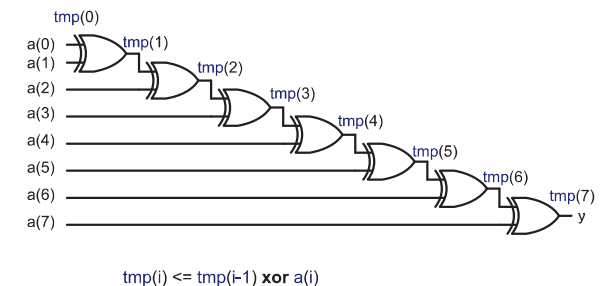
```
library ieee;
use ieee.std_logic_1164.all;
entity generic_demo is
  port(
    clk, reset: in std_logic;
    q_4: out std_logic_vector(3 downto 0);
    q_12: out std_logic_vector(11 downto 0)
  );
end entity generic_demo;
```

265

```
architecture arch of generic_demo is
  component para_binary_counter is
    generic (WIDTH: natural);
    port (
      clk, reset: in std_logic;
      q: out std_logic_vector(WIDTH-1 downto 0)
    );
  end component para_binary_counter;
begin
  four_bit: para_binary_counter
    generic map (WIDTH => 4)
    port map (clk => clk, reset => reset, q => q_4);
  twelve_bit: para_binary_counter
    generic map (WIDTH => 12)
    port map (clk => clk, reset => reset, q => q_12);
end architecture arch;
```

266

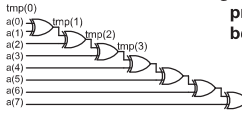
Example: Reduced-xor circuit



267

Parameterized reduced-xor circuit using a generic

```
library ieee; use ieee.std_logic_1164.all;
entity reduced_xor is
  generic (WIDTH: natural); -- generic declaration
  port(
    a: in std_logic_vector(WIDTH-1 downto 0);
    y: out std_logic
  );
end entity reduced_xor;
architecture loop_linear_arch of reduced_xor is
  signal tmp: std_logic_vector(WIDTH-1 downto 0);
begin
  process (a, tmp) is
  begin
    tmp(0) <= a(0); -- boundary bit
    for i in 1 to (WIDTH-1) loop
      tmp(i) <= a(i) xor tmp(i-1);
    end loop;
    end process;
    y <= tmp(WIDTH-1);
  end architecture loop_linear_arch;
```



Slide 280

Slide 293

268

8.2 Array attribute

- A VHDL **attribute** provides information about a named item, such as a data type or a signal.
- We have used the 'event attribute, as in **clk'event**, express the changing edge of the **clk** signal.
- A set of attributes is associated with an object of an **array** data type. Let **s** be a signal with an array data type.
 - s'left**, **s'right**: the left and right bounds of the index range of **s**.
 - s'low**, **s'high**: the lower and upper bounds of the index range of **s**.
 - s'length**: the length of the index range of **s**.
 - s'range**: the index range of **s**.
 - s'reverse_range**: the reversed index range of **s**.

269

- The attributes can be applied to the signal defined with **std_logic_vector**, unsigned and signed:
 - signal s1**: **std_logic_vector** (31 **downto** 0);
 - signal s2**: **std_logic_vector** (8 **to** 15);

The attributes of **s1** are

s1'left = 31; **s1'right** = 0;
s1'low = 0; **s1'high** = 31;
s1'length = 32;
s1'range = 31 **downto** 0
s1'reverse_range = 0 **to** 31

The attributes of **s2** are

s2'left = 8; **s2'right** = 15;
s2'low = 8; **s2'high** = 15;
s2'length = 8;
s2'range = 8 **to** 15
s2'reverse_range = 15 **downto** 8

270

Parameterized reduced-xor circuit using an attribute

```
architecture loop_linear_arch of reduced_xor is
  signal tmp: std_logic_vector(a'length-1 downto 0);
begin
  process (a, tmp) is
  begin
    tmp(0) <= a(0);
    for i in 1 to (a'length-1) loop
      tmp(i) <= a(i) xor tmp(i-1);
    end loop;
    end process;
    y <= tmp(a'length-1);
  end architecture loop_linear_arch;
```

```
library ieee;
use ieee.std_logic_1164.all;
entity reduced_xor is
  generic(W: natural);
  port(a: in std_logic_vector(W-1 downto 0);
        y: out std_logic
  );
end entity reduced_xor;
```

- The range of the for loop can also be expressed as:
 - for i in a'low+1 to a'high loop**
 - for i in a'right+1 to a'left loop**
- The last signal assignment
 - y <= tmp(tmp'left);**

271

8.3 Unconstrained Array

- Unconstrained array is defined as an array type with specified data type of the index value, but without specified exact bounds of the index value.
- Example:


```
type std_logic_vector is array (natural range <>) of std_logic
```
- Similarly, we have **unsigned** and **signed** data types.
- If an object is declared with an unconstrained array data type, we must specify its index range when the data type is used, as 15 **downto** 0 in


```
signal x: std_logic_vector(15 downto 0);
```

272

- A special case: the unconstrained array can be declared without specifying the range in port declaration.
- Example:

```
library ieee; use ieee.std_logic_1164.all;
entity unconstrain_dff is
  port( clk: std_logic;
        d: in std_logic_vector; -- the actual range is inferred
        q: out std_logic_vector -- when an instance of unconstrain_dff is instantiated.
  );
end entity unconstrain_dff;
architecture arch of unconstrain_dff is
begin
  process (clk) is
  begin
    if (clk'event and clk='1') then q <= d; end if;
  end process;
end architecture arch;
```

273

- The ranges of the actual signals become the ranges of **d** and **q** signals. Example: the **dff16** instance is instantiated as a 16-bit register

```
...
signal din, qout: std_logic_vector(15 downto 0);
signal clk: std_logic;
...
dff16: unconstrain_dff
  port map( clk => clk, d => din, q => qout);
...
```

- Since no range is specified for **d** and **q**, the boundaries of the two signal will not be checked in the analysis stage.

```
...
signal din: std_logic_vector(15 downto 0);
signal qout: std_logic(7 downto 0); -- syntactically correct.
...
-- error may be detected during the synthesis
dff16: unconstrain_dff
  port map( clk => clk, d => din, q => qout);
...
```

274

Parameterized reduced-xor circuit using an unconstrained array

The code appears to be correct at first glance

```
library ieee; use ieee.std_logic_1164.all;
entity unconstrain_reduced_xor is
  port(
    a: in std_logic_vector;
    y: out std_logic
  );
end entity unconstrain_reduced_xor;
architecture arch of unconstrain_reduced_xor is
  constant WIDTH: natural := a'length;
  signal tmp: std_logic_vector(WIDTH-1 downto 0);
begin
  process (a, tmp) is
  begin
    tmp(0) <= a(0);
    for i in 1 to (WIDTH-1) loop
      tmp(i) <= a(i) xor tmp(i-1);
    end loop;
    end process;
    y <= tmp(WIDTH-1);
  end architecture arch;
```

potential error:
 如果a信号是15 downto 8
 where is a(0)?

275

- If we map the a signal to an actual signal with the type of **std_logic_vector** of 8 bits during component instantiation, we may have a to be:
 - std_logic_vector**(7 **downto** 0);
 - std_logic_vector**(0 **to** 7);
 - std_logic_vector**(15 **downto** 8);
 - std_logic_vector**(8 **to** 15);
- The code does not work properly for the last two formats.

276

Improved
parameterized
reduced-xor
circuit using
an
unconstrained
array

```
architecture better_arch of unconstrain_reduced_xor is
    constant WIDTH: natural := a'length;
    signal tmp: std_logic_vector(WIDTH-1 downto 0);
    signal aa: std_logic_vector(WIDTH-1 downto 0);
begin
    aa <= a;
    process (aa, tmp) is
    begin
        tmp(0) <= aa(0);
        for i in 1 to (WIDTH-1) loop
            tmp(i) <= aa(i) xor tmp(i-1);
        end loop;
    end process;
    y <= tmp(WIDTH-1);
end architecture better_arch;
```

277

8.3 Comparison

- The unconstrained array mechanism uses attributes to infer the relevant information from the actual signal.
 - More general and flexible than the generic mechanism, but also
 - More opportunities for errors.
 - Requires comprehensive error-checking code
- Generic mechanism is preferred, unless a module is extremely general and widely used.
 - More rigid
 - It clearly specifies the range, direction and width of each signal and avoids many subtle erroneous conditions.

278